

Progress

AI-RAN GPU Isolation — NSYS Time Decomposition Deep-Dive

Changjong Kim

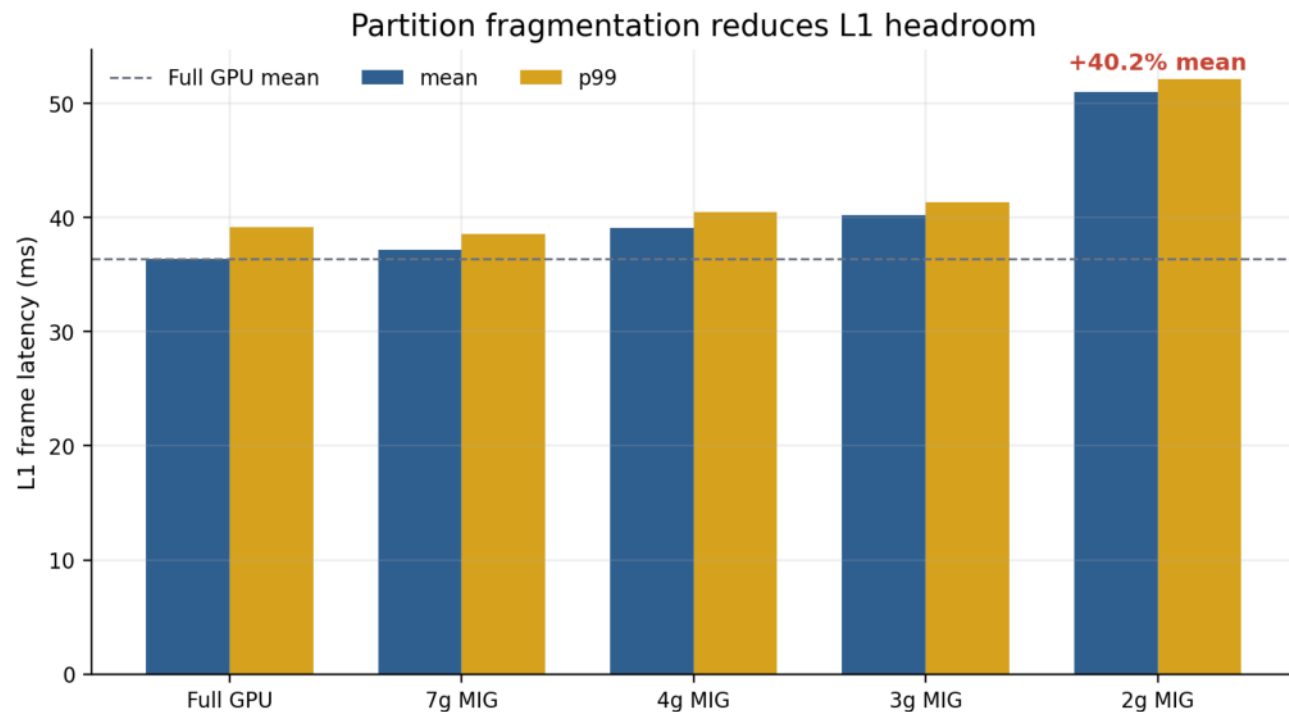
Bigdata and HPC Lab

Department of Computer Science and Engineering

Seoul National University of Science and Technology

Today

- Continuation of Exp1-6 (5/29): MIG single-tenant overhead +13-32 ms measured.
- Today's approach: Profile L1 + AI co-tenancy with NSYS time-decomposition analysis.
- Flow: cuPHY L1 background → NSYS decomposition (GPU/host/pipeline) → queue mechanism → verification experiments → Perlmutter default → MPS introduces a paradox → cudaFree unifies the whole story.

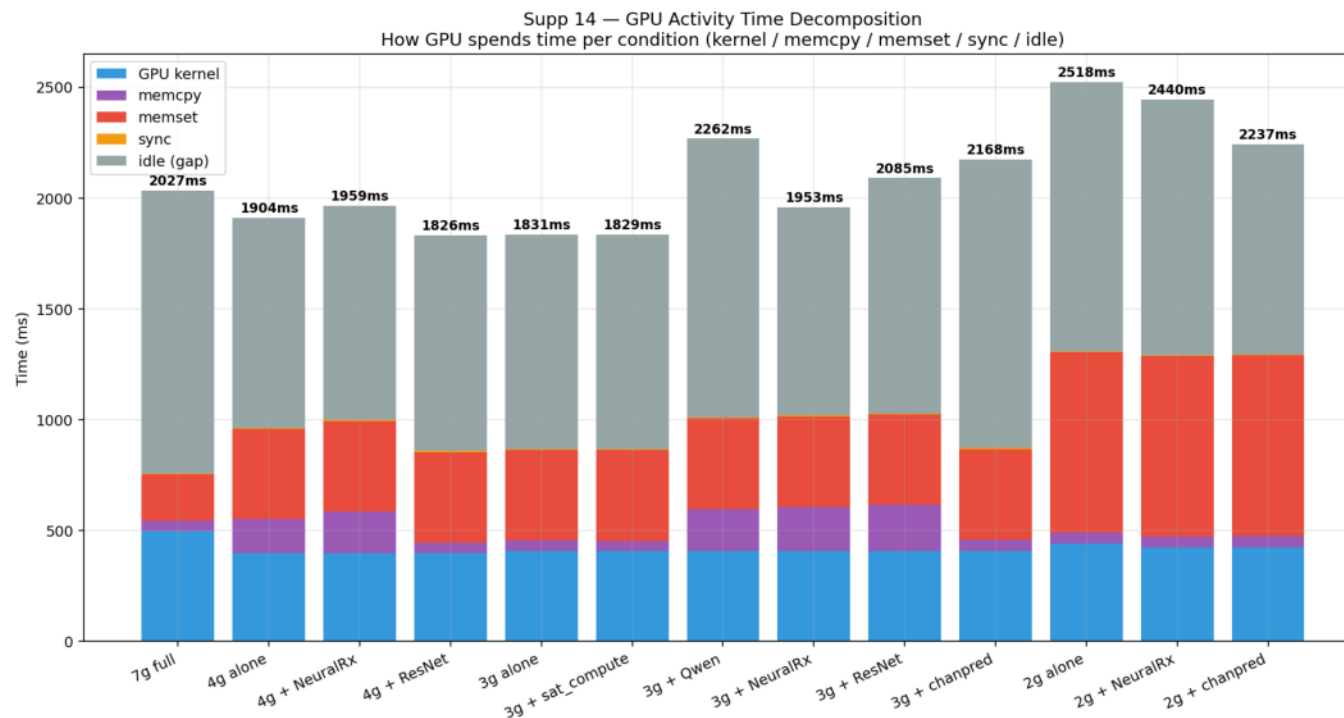


cuPHY L1 — What We Measure

- **Pipeline:** PUSCH RX = pre-ChEst → ChEst dispatch → Noise/Intf estimation → MMSE equalizer → LDPC decode, with ``convert_kernel`` between stages.
- **Per-frame setup:** 20 cells serial, MCS=2, 273 PRB, 4×4 antennas. n=500-1000 per condition.
- **Decomposition target:** L1 frame latency p99 + NSYS GPU/host trace broken into kernel / memcpy / memset / sync / idle gap and CUDA runtime API per call.

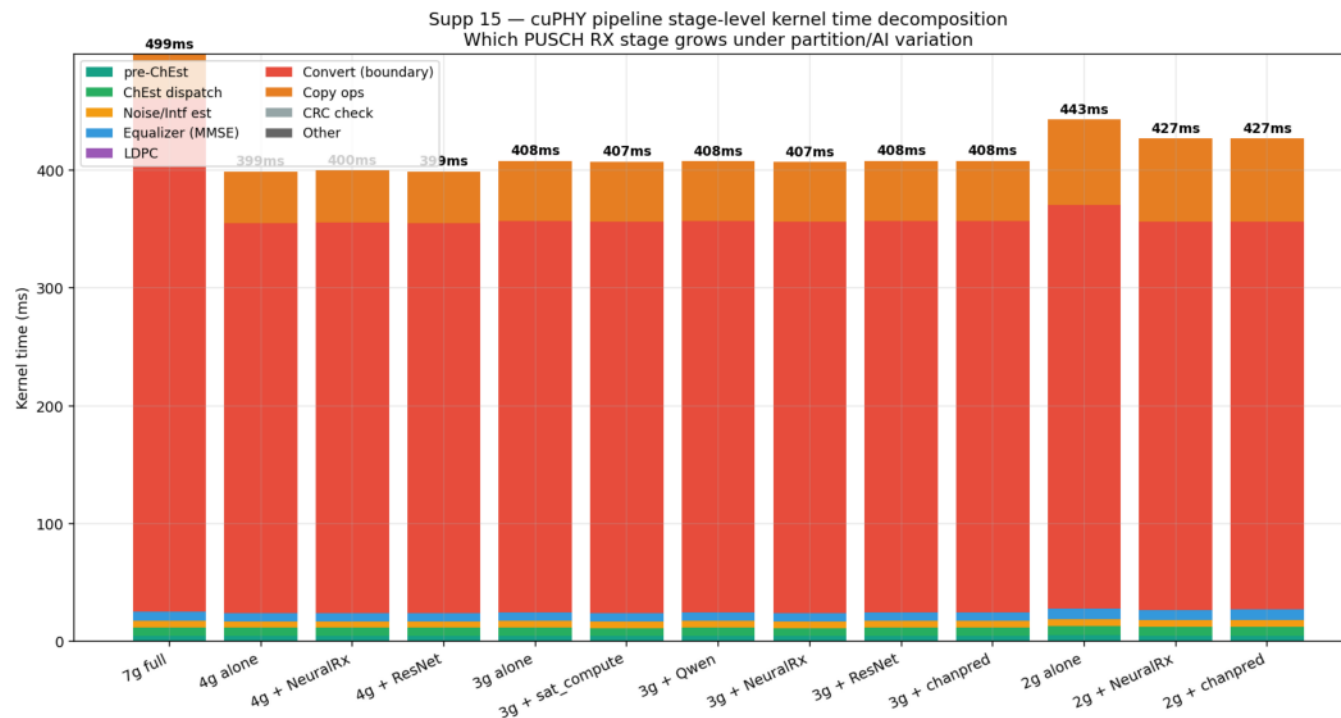
NSYS §17.1 — GPU Activity Decomposition

- Wall-clock varies 1.4×: 7g 2,027 ms, 3g alone 1,831 ms, 2g alone 2,518 ms — same cuPHY work, different total time.
- GPU kernel time is invariant: ~400-500 ms in every condition. AI co-location does not slow L1 compute itself.
- Idle gap dominates 40-60% of wall-clock: The cost is in waiting between operations, not in computing.



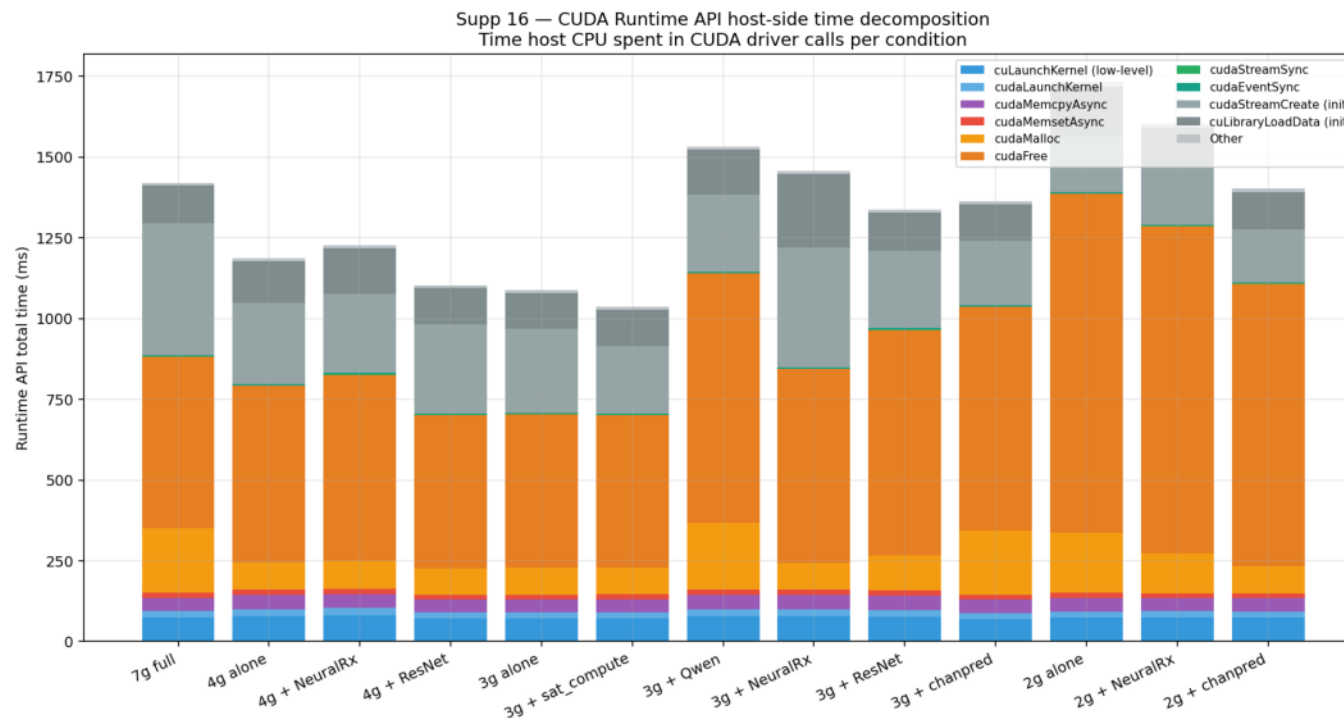
NSYS §17.2 — cuPHY Pipeline Stage Breakdown

- Per-stage time is small: ChEst, Noise/Intf, Equalizer, LDPC all 5-10 ms each.
- `convert_kernel` dominates 357 ms: A boundary op between pipeline stages, called $\sim 4\times$ per cell iteration (2,576 calls / 640 iter).
- Stage timing is partition/AI-invariant: L1 kernel work is isolated as expected; the cost lives at the stage boundary, not in the stages.



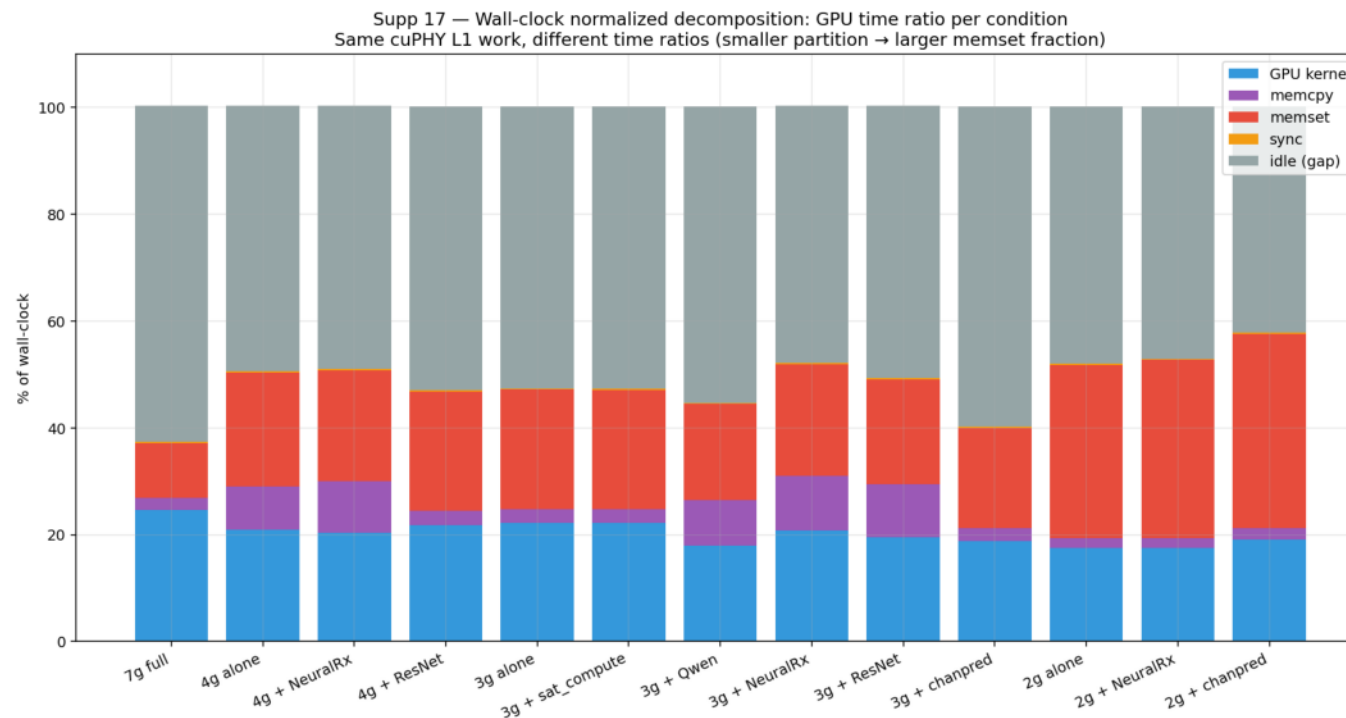
NSYS §17.3 — Host CPU Runtime API

- `cudaFree` is the largest host cost: 598 ms (1st). cuPHY allocates per-frame and calls `cudaFree` ~1,983 times per measurement.
- `cuLaunchKernel` is invariant ~80 ms: Launch queue is not contended; the host-side overhead is in memory management, not in kernel launch.
- Small partition compounds the cost: `cudaFree` on 2g = 1,380 ms vs 7g = 880 ms (1.6×). Driver's page-table management scales with partition size.



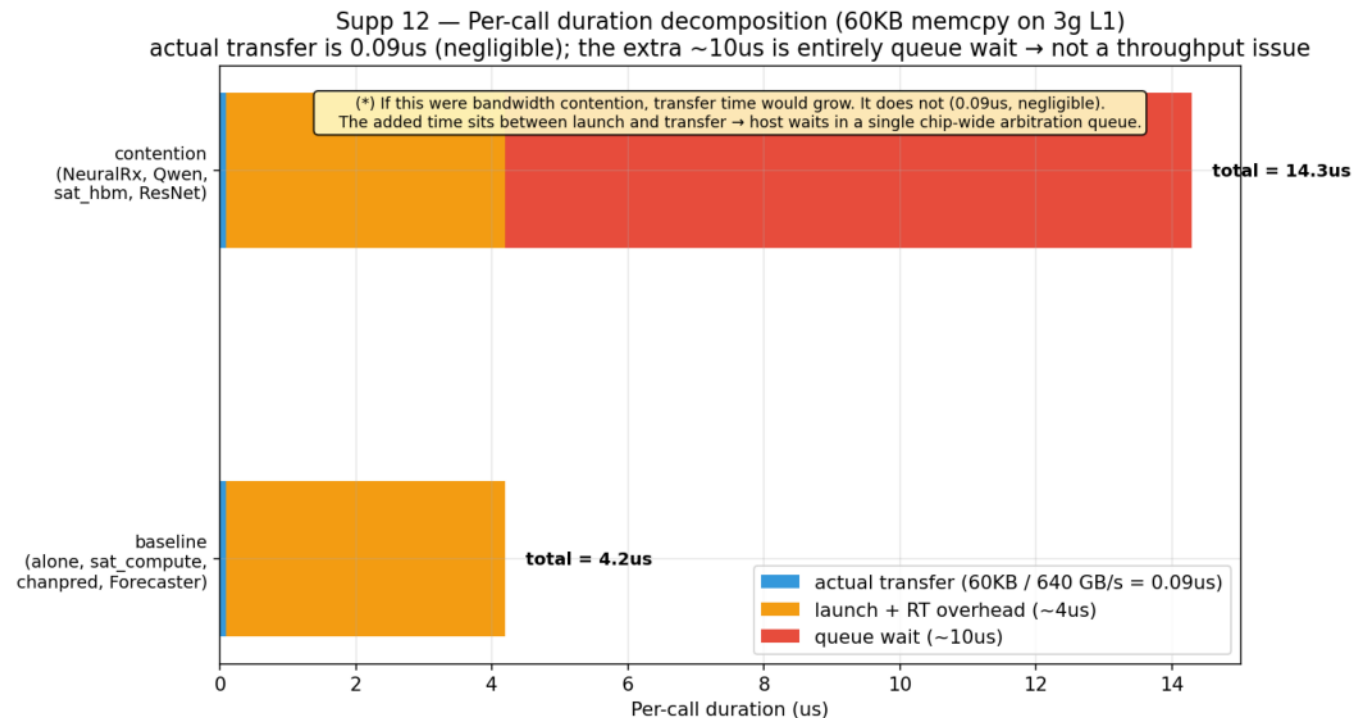
NSYS §17.4 — Wall-Clock Normalized Decomposition

- GPU kernel: ~20% in every condition. L1 compute is a small slice of wall-clock; the rest is overhead.
- memset grows with smaller partition: 7g 10% → 3g 19% → 2g 36%. Structural HBM bandwidth share scaling, visible as memset time.
- idle gap shrinks with smaller partition (63% → 42%) but wall-clock still grows: Memset displaces idle as the bottleneck, so smaller partition still loses on total latency.



Mechanism — Arithmetic Proves Queue Wait, Not Throughput

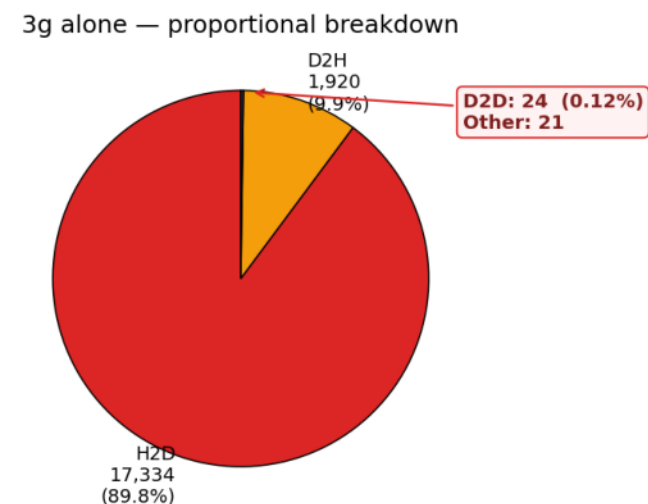
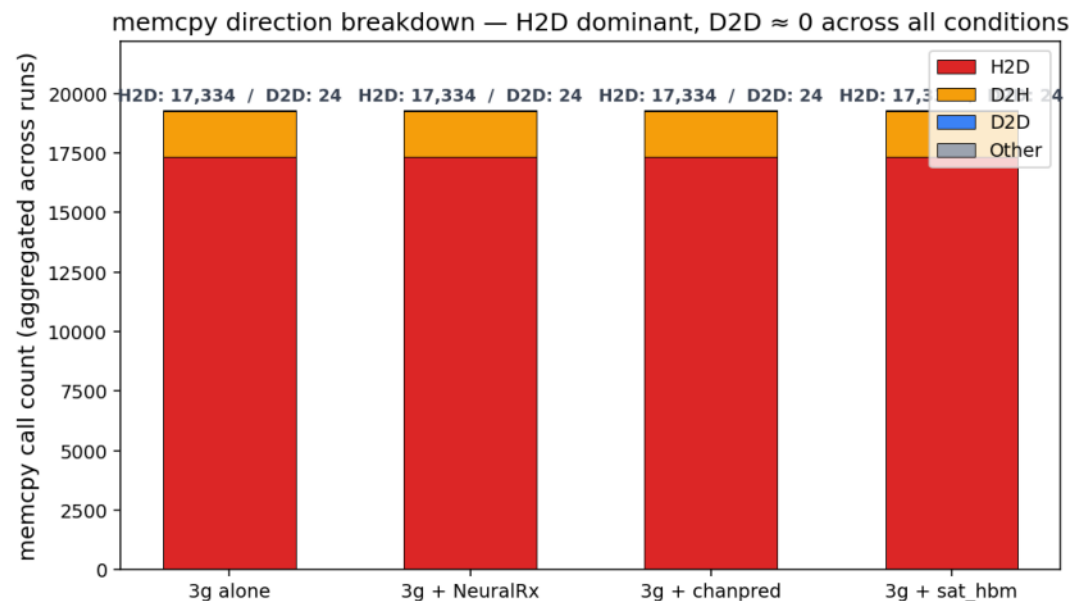
- Actual transfer is negligible: $60 \text{ KB} / 640 \text{ GB/s} = 0.09 \mu\text{s}$. Bandwidth saturation cannot account for the inflation.
- Baseline: $4.2 \mu\text{s}$ alone = launch + runtime overhead; transfer contributes essentially nothing.
- Contended: $14.3 \mu\text{s} = 4.2 \mu\text{s}$ baseline + $\sim 10 \mu\text{s}$ queue wait. The slow mode is queue wait by construction.



Mechanism — Queue Located at Chip-Wide PCIe/DMA Copy Engine

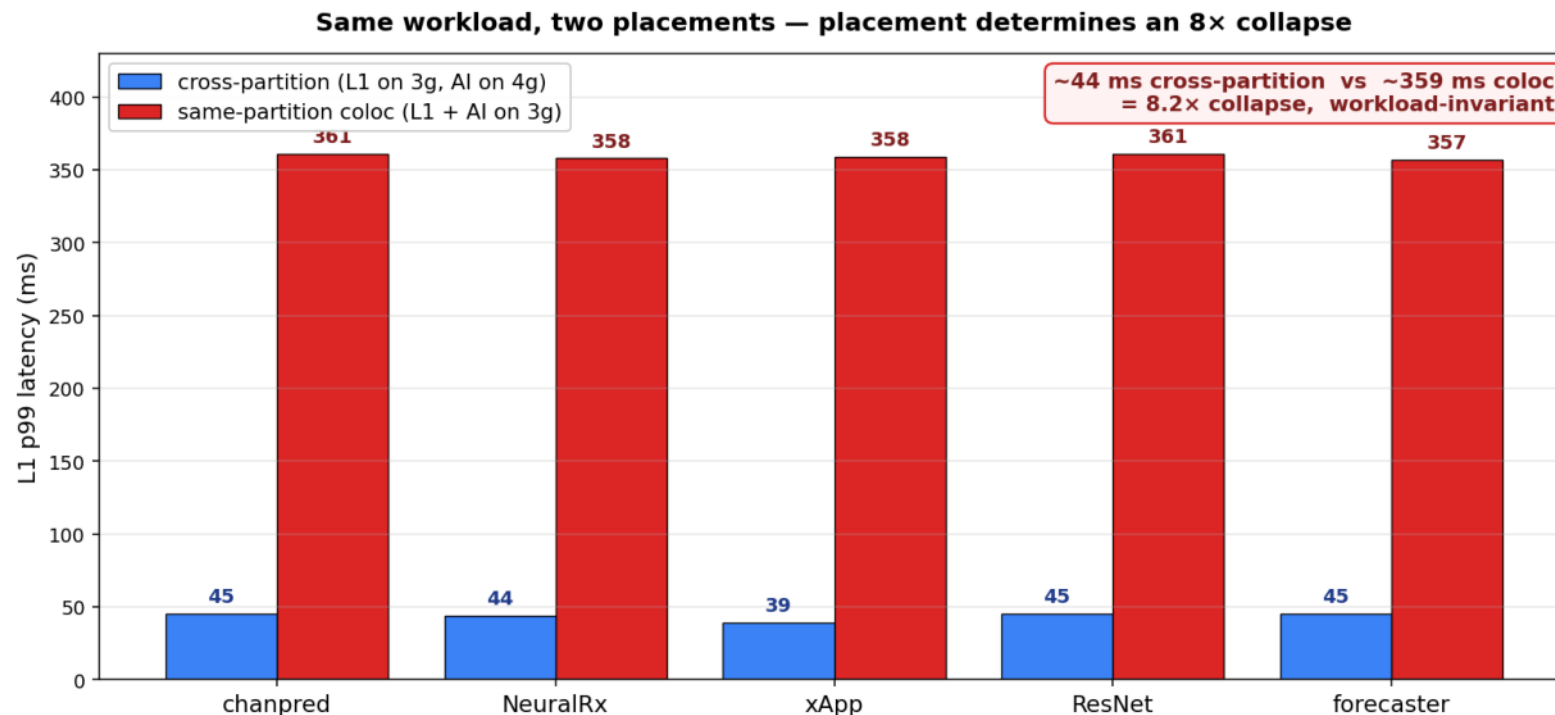
- **Direction breakdown:** 89.8% H2D, 9.9% D2H, 0.12% D2D across all measured memcpy calls.
- **D2D ≈ 0 rules out the HBM controller:** Which would carry D2D traffic.
- **Single shared resource:** The PCIe/DMA copy engine is one physical unit serving the entire chip; partitioning cannot isolate it.

Queue location proof — memcpy direction: 100% H2D, D2D ≈ 0
→ Queue is at chip-wide PCIe/DMA copy engine (not partition-isolated HBM controller)



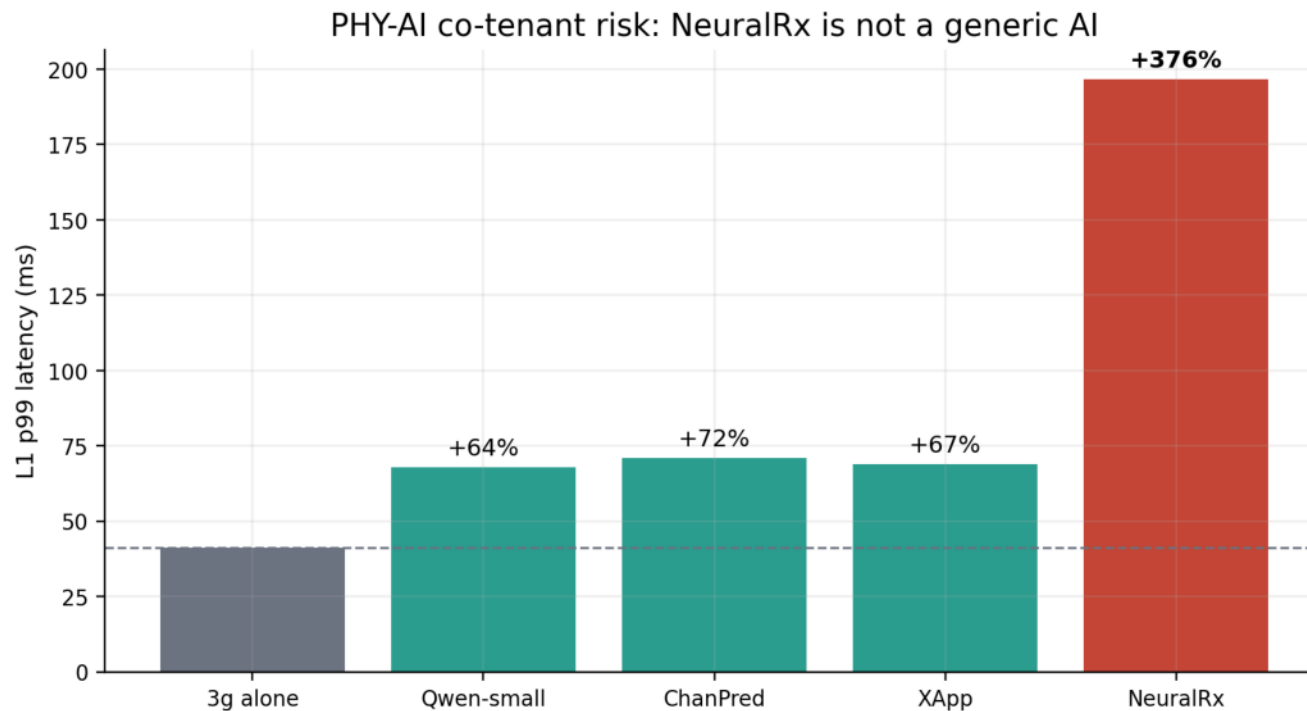
Verification 1 — Placement Determines an 8× Collapse

- **Cross-partition is stable:** Five distinct AI workloads (chanpred, NeuralRx, xApp, ResNet, forecaster) all keep L1 p99 at ~45 ms on a separate MIG partition.
- **Same-partition collapses 8×:** Identical workloads co-located in the same partition jump to ~358 ms.
- **Confirms the queue hypothesis:** The contended resource is placement-sensitive but workload-invariant, as predicted by a single chip-wide queue.



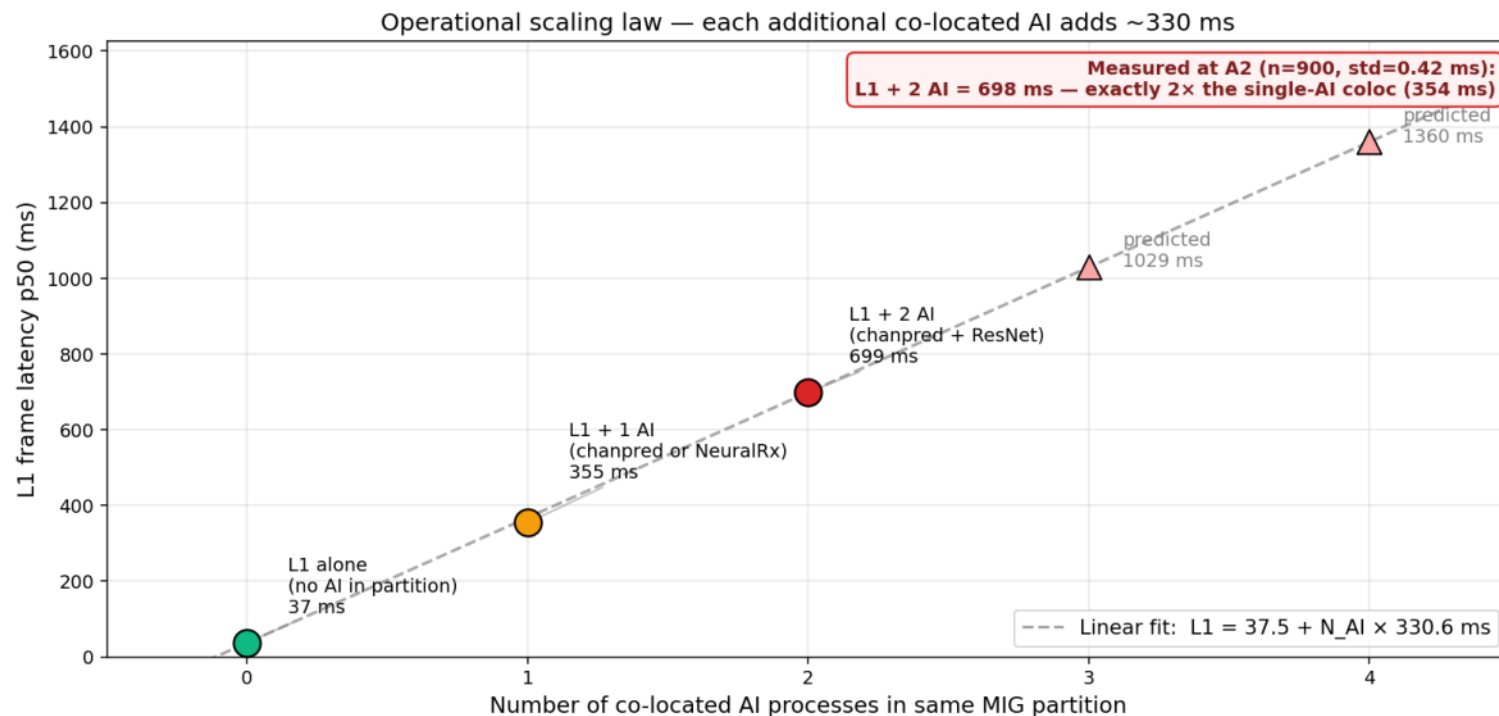
Verification 2 — NeuralRx PHY-AI Uniqueness

- **PHY-AI breaks cross-partition isolation:** NeuralRx in a separate partition still inflates L1 p99 by +376% (41 → 197 ms); ChanPred, xApp, Qwen on the same setup only inflate L1 by +60-72%.
- **Why NeuralRx specifically:** NeuralRx (TensorRT inline NN) shares the cuPHY-style copy/convert traffic pattern with L1, producing contention even across MIG partition boundaries.
- **Implication:** Even cross-partition placement does not isolate L1 from PHY-AI co-tenants.



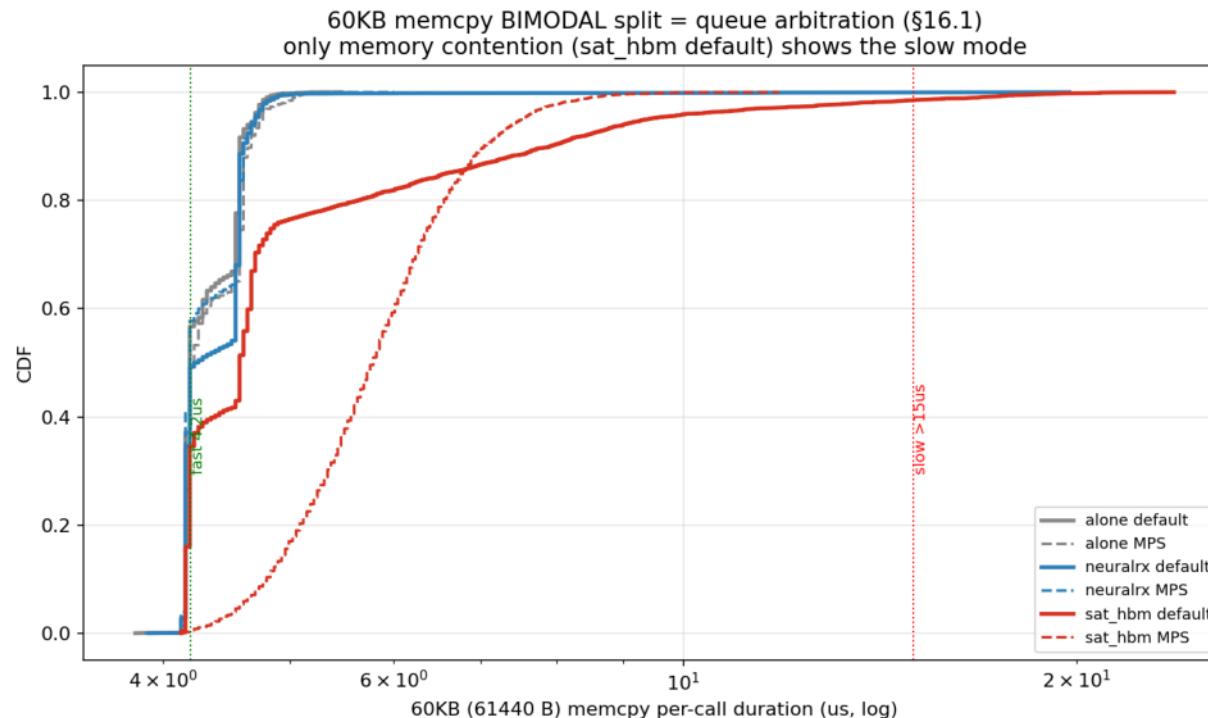
Verification 3 — N-AI Operational Scaling Law

- **Linear cost model:** $L1 = 37.5 + N_AI \times 330.6$ ms when AI processes share the same MIG partition.
- **A2 validates the slope:** L1 + 2 AI measures 698 ms (n=900, std=0.42 ms) — exactly 2× the single-AI co-location (354 ms).
- **Multi-tenant mistakes compound:** Predicted 1,029 ms at 3 AI and 1,360 ms at 4 AI. Operational error scales linearly, not as a single step.



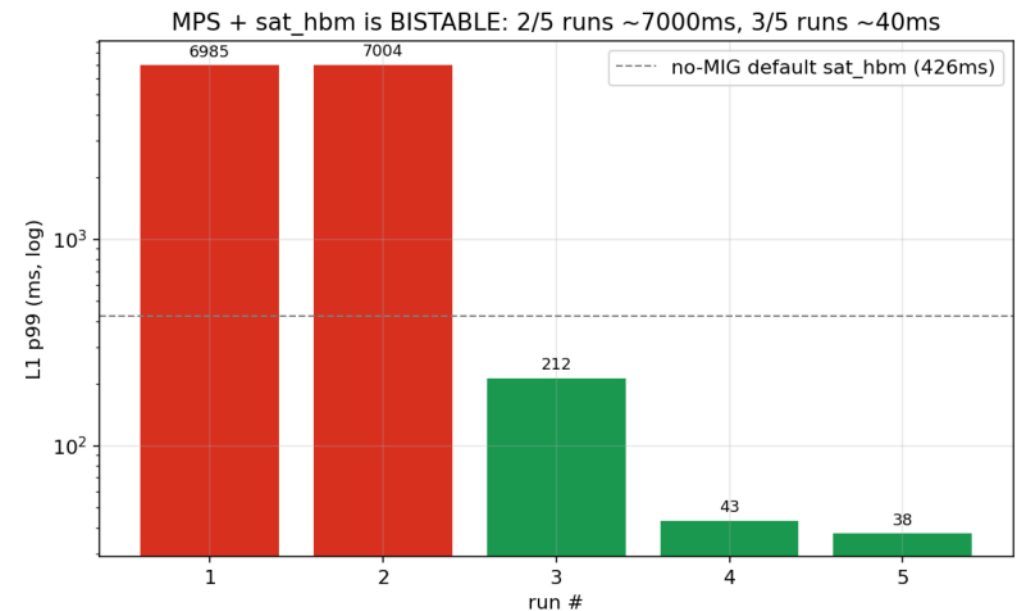
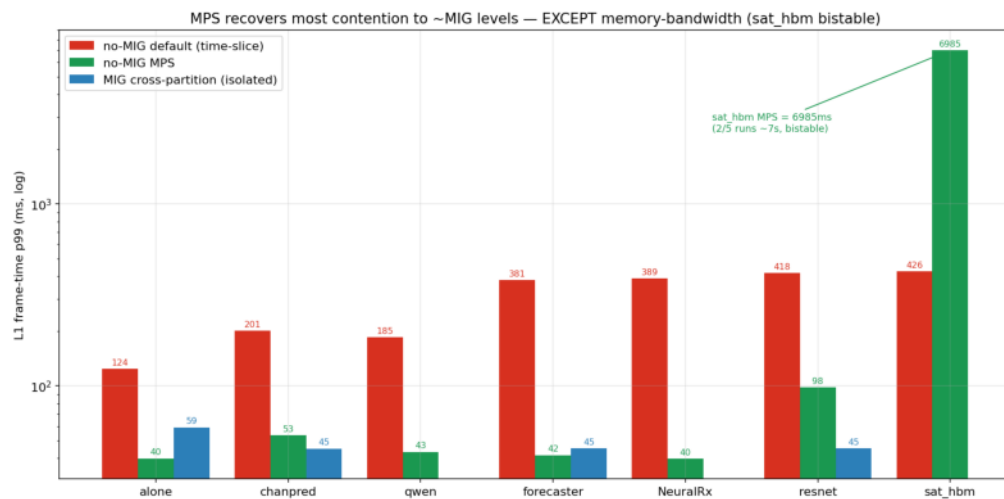
Cross-Platform Validation on Perlmutter (Default Mode)

- **Bimodal signature reproduced:** Perlmutter no-MIG (different cluster, different driver) shows the same 60 KB memcpy bimodal split ($4.2 \leftrightarrow 16.8 \mu\text{s}$) under L1 + AI co-tenancy.
- **cudaFree growth matches CloudLab:** under NeuralRx default the Perlmutter cudaFree average grows $15\times$ — identical fold-change to the CloudLab MIG measurement.
- **Conclusion:** the queue + cudaFree mechanism is NVIDIA-stack-wide, not specific to MIG or the CloudLab driver — same default-mode signature on independent hardware.



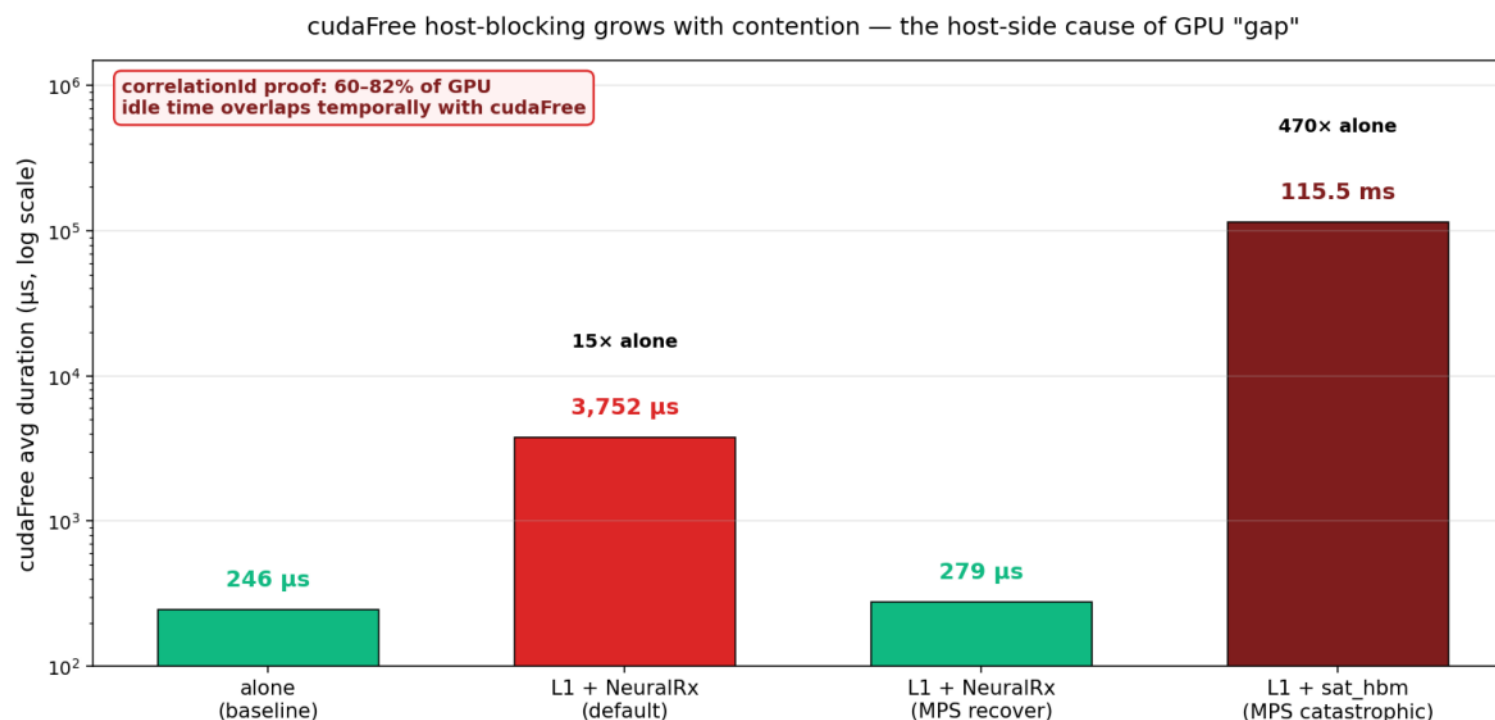
MPS Recovers Compute AI, Catastrophic for Memory AI

- **Compute AI recovers:** NeuralRx 389 → 40 ms under MPS (below MIG cross-partition 197 ms); forecaster 381 → 42 ms; Qwen 185 → 43 ms.
- **Memory AI becomes worse:** sat_hbm 426 → 6,985 ms under MPS (bistable across runs).
- **Open question:** why does MPS help compute-bound AI but break memory-bound AI? The next slide answers via the cudaFree mechanism.



Mechanism — cudaFree Host-Blocking Explains Both MIG and MPS

- **Default-mode mechanism:** alone 246 μ s $\rightarrow$ L1 + NeuralRx default 3,752 μ s (15 $\times$ alone). The host blocks on cudaFree; correlationId shows 60-82% of the GPU idle gap overlaps with cudaFree on the host.
- **MPS recovery explained:** L1 + NeuralRx under MPS drops cudaFree back to 279 μ s ($\approx$ alone), so the GPU stops idling — recovery is the cudaFree path being un-blocked, not a new isolation mechanism.
- **MPS catastrophic explained:** L1 + sat_hbm under MPS pushes cudaFree to 115,506 μ s (470 $\times$ alone). Memory-bound co-tenants starve the device of free-able memory, so cudaFree blocks longer — same mechanism, opposite outcome.
- **Unifying conclusion:** kernel-invariant + gap-dominated + cudaFree-dominated decomposition from §17 is the host-blocking story all the way through — MIG, MPS recover, MPS catastrophic are three points on the same curve.



Design Rules

- **Placement enforcement is mandatory:** L1 and AI must reside in different MIG partitions; one misplacement produces the full 8× collapse.
- **Partition sizing serves only L1 budget:** Sizing has no mitigating effect under co-location; choose for L1 SM/HBM headroom only.
- **AI workload classification is required:** Memory-bound AI cannot be safely co-located even under MPS.
- **MIG $\neq$ chip-wide isolation:** MIG isolates per-partition SM/HBM but not the chip-global PCIe/DMA copy engine.

Thank You
